¿Qué es un Socket?



Un socket es una interfaz de software que permite la comunicación entre procesos, ya sea en la misma máquina o en diferentes máquinas a través de una red.

Los sockets proporcionan un mecanismo para que las aplicaciones envíen y reciban datos entre sí, y se utilizan ampliamente en la programación de redes para habilitar la comunicación entre servidores y clientes.

Conceptos básicos de un socket

Un socket se puede imaginar como un punto de conexión entre dos aplicaciones (o procesos) que quieren intercambiar información. Existen diferentes tipos de sockets, dependiendo del tipo de comunicación y del protocolo de red utilizado, pero el más comúnmente usado es el socket de red basado en el protocolo TCP/IP.



Componentes principales de un socket

Dirección IP

La dirección única que identifica un dispositivo en la red (por ejemplo, 192.168.1.1 en una red local o www.example.com en Internet).

Puerto

El número que identifica una aplicación o servicio específico dentro de un dispositivo. Los puertos son números enteros (por ejemplo, el puerto 80 se usa para servidores HTTP, 443 para HTTPS, 3306 para MySQL, etc.).

Protocolo de comunicación

Determina cómo se va a transmitir la información entre los procesos. Los protocolos más comunes son:

TCP (Transmission Control Protocol): Protocolo orientado a la conexión que garantiza la entrega de los datos de manera fiable, secuencial y sin errores.

Tipos de sockets

Existen diferentes tipos de sockets que se utilizan según el protocolo y la naturaleza de la comunicación

1 **Socket de flujo (Stream Socket)**

- Utiliza el protocolo TCP para la comunicación orientada a la conexión.
- Los datos se envían en una secuencia continua de bytes. Este tipo de socket garantiza que los datos lleguen en el mismo orden en que fueron enviados.
- Se usa principalmente en aplicaciones que requieren una comunicación fiable, como navegadores web, correo electrónico o transferencia de archivos.

2 **Socket de datagramas (Datagram Socket)**

- Utiliza el protocolo UDP, que es un protocolo sin conexión.
- No garantiza la entrega de los mensajes ni su orden, pero es más rápido y eficiente para ciertas aplicaciones que necesitan enviar datos sin preocuparse por los errores, como streaming de video, videojuegos online o servicios de DNS.

3 **Socket UNIX (Unix Domain Socket)**

- Utilizado para la comunicación entre procesos en la misma máquina, sin necesidad de una red. Es una forma eficiente de intercambiar datos entre procesos locales (en lugar de usar redes TCP/IP).

4 **Socket de transmisión confiable (Reliable Datagram Socket)**

- Se utiliza en aplicaciones que necesitan transmitir datos de manera confiable pero no requieren los servicios completos de TCP.



Cómo funciona un socket

Creación

Un socket se crea usando una llamada a una función del sistema operativo (como socket() en C o socket.socket() en Python). El tipo de socket se especifica en este paso (por ejemplo, un socket TCP o UDP).

Conexión (solo para TCP)

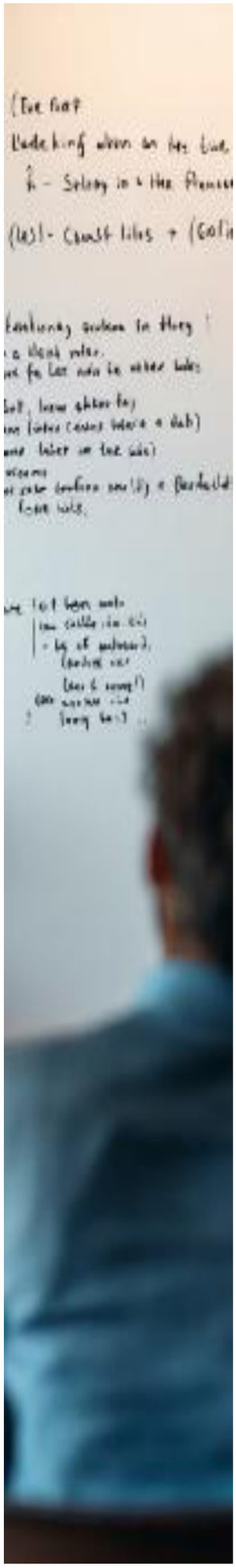
En el caso de los sockets TCP (orientados a la conexión), el cliente debe conectarse a un servidor mediante una llamada a connect(), mientras que el servidor debe esperar conexiones con bind() y listen() antes de aceptar conexiones con accept(). En los sockets UDP, no es necesario establecer una conexión, ya que se trabaja en un modelo sin conexión.

Envío y recepción de datos

Una vez establecida la conexión, los sockets permiten enviar y recibir datos mediante las funciones send() y recv() (en TCP), o sendto() y recvfrom() (en UDP).

Cierre

Una vez que la comunicación ha terminado, el socket debe cerrarse mediante la llamada close(), liberando los recursos del sistema.



Ejemplo básico de uso de un socket

Servidor (Socket TCP)

Código en Python

```
import socket
```

Crear un socket TCP (SOCK_STREAM):

```
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

Vincular el socket a una dirección IP y puerto:

```
server_socket.bind(('localhost', 8080))
```

Escuchar por conexiones entrantes

```
server_socket.listen(1)
print("Esperando conexión de un cliente...")
```

Aceptar la conexión

```
client_socket, client_address = server_socket.accept()
print(f"Conexión establecida con {client_address}")
```

Enviar un mensaje al cliente

```
client_socket.send(b"Hola, cliente! Bienvenido al servidor.")
```

Recibir datos del cliente

```
data = client_socket.recv(1024)
print(f"Recibido del cliente: {data.decode()}")
```

Cerrar la conexión

```
client_socket.close()
server_socket.close()
```

Cliente (Socket TCP)

Código en Python

```
import socket
```

Crear un socket TCP (SOCK_STREAM):

```
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

Vincular el socket a una dirección IP y puerto:

```
server_socket.bind(('localhost', 8080))
```

Recibir un mensaje del servidor

```
message = client_socket.recv(1024)
print(f"Recibido del servidor: {message.decode()}")
```

Enviar datos al servidor

```
client_socket.send(b"Hola, servidor! Soy el cliente.")
```

Cerrar la conexión

```
client_socket.close()
```

En este ejemplo, el servidor y el cliente se comunican a través de un socket TCP en la dirección localhost y el puerto 8080. El servidor espera una conexión, envía un mensaje al cliente y luego espera una respuesta. El cliente se conecta al servidor, recibe el mensaje y luego responde.

Aplicaciones de los sockets



Navegación web (HTTP/HTTPS)

Los navegadores web utilizan sockets para conectarse a los servidores web y cargar páginas.

Correo electrónico (SMTP/POP/IMAP)

Los servidores y clientes de correo electrónico utilizan sockets para enviar y recibir mensajes de correo.

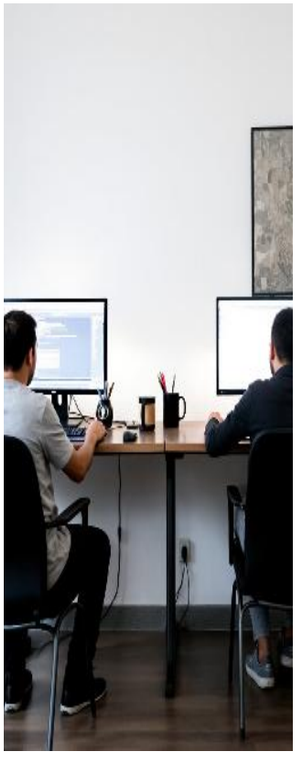
Juegos en línea

Los sockets permiten la comunicación entre clientes (jugadores) y servidores en tiempo real.

Aplicaciones de mensajería

Plataformas como WhatsApp, Telegram y Slack utilizan sockets para la comunicación en tiempo real.

Ejercicios para la clase 5



Imagina que estás creando una pequeña aplicación de chat usando sockets en Python. El sistema tendrá dos partes:

1. El cliente que envía un mensaje al servidor.

2. El servidor que recibe el mensaje y responde al cliente con un mensaje de confirmación.

• Crea un pequeño servidor que escuche en un puerto específico y reciba mensajes de los clientes.

• Crea un cliente que se conecte al servidor, envíe un mensaje y luego imprima la respuesta del servidor.